

Scrapy — это высокоуровневый фреймворк для веб-краулинга и извлечения структурированных данных из веб-страниц, написанный на Python и предназначенный для решения широкого спектра задач: от интеллектуального анализа данных до мониторинга и автоматизированного тестирования. В отличие от простых скриптов, Scrapy предоставляет полноценную проектную структуру, которая организует пауков, настройки, элементы и конвейеры, позволяя решению масштабироваться от одного паука до крупных сопровождаемых проектов. Фреймворк построен на основе Twisted — популярной событийно-ориентированной сетевой библиотеки для Python, что обеспечивает неблокирующую асинхронную обработку запросов и эффективный параллелизм без необходимости управлять потоками вручную.

Scrapy's building blocks

Scrapy splits the work of crawling and scraping across a handful of pieces, each responsible for one part of the problem. Placing a given piece of custom logic where it belongs keeps that logic reusable across spiders and the project easy to follow — this separation is what lets Scrapy scale from a single spider to large, maintainable projects, rather than one script per site.

This page is a quick map of those pieces, what problem each one solves, and when to reach for it.

Spiders

A **spider** defines which requests to send and how to parse their responses. It is where a scraping project usually starts: one spider per site (or group of similar sites), yielding **items** and further requests as it parses each response.

Items

An **item** is the data you extract, as a plain key-value object. Spiders yield them, and everything downstream (item pipelines, feed exports) works with them regardless of which item type a project uses.

Settings

Settings configure everything else on this page: which middlewares and extensions are enabled and in what order, how item pipelines behave, feed export destinations, and Scrapy's own defaults. Settings are usually project-wide, but a spider can override them for itself alone through `custom_settings`, which is often the simplest way to turn something project-wide, such as a downloader middleware, into behavior that only applies to the spiders that need it.

Installing Scrapy

If you're using **Anaconda** or **Miniconda**, you can install the package from the **conda-forge** channel, which has up-to-date packages for Linux, Windows and macOS.

To install Scrapy using **conda**, run:

```
conda install -c conda-forge scrapy
```

Alternatively, if you're already familiar with installation of Python packages, you can install Scrapy and its dependencies from PyPI with:

```
pip install Scrapy
```

BASIC CONCEPTS
Scrapy's building blocks
Command line tool
Spiders
Selectors
Items
Item Loaders
Scrapy shell
Item Pipeline
Feed exports
Requests and Responses
Cookies
Link Extractors

Архитектура Scrapy организована вокруг нескольких ключевых компонентов, взаимодействие между которыми управляется движком. Движок (Engine) отвечает за контроль потока данных между всеми частями системы и инициирует события при выполнении определённых действий. Планировщик (Scheduler) получает запросы от движка, ставит их в очередь и возвращает движку, когда тот запрашивает следующую порцию для обхода. Загрузчик (Downloader) выполняет выборку веб-страниц и передаёт их движку, который, в свою очередь, направляет их паукам для обработки. Пауки (Spiders) представляют собой пользовательские классы, написанные для разбора ответов и извлечения из них элементов или дополнительных запросов для дальнейшего обхода. Конвейеры элементов (Item Pipelines) обрабатывают извлечённые элементы, выполняя такие типичные задачи, как очистка, валидация и сохранение в базу данных. Кроме того, в архитектуре предусмотрены промежуточные слои: промежуточное ПО загрузчика (Downloader Middlewares) и промежуточное ПО пауков (Spider Middlewares), которые позволяют внедрять пользовательскую логику на уровне обработки запросов и ответов, не изменяя код паука.

Поток данных в Scrapy управляется движком и следует чётко определённой последовательности. Движок получает начальные запросы от паука и передаёт их планировщику, который ставит их в очередь и возвращает движку следующую порцию для обхода. Движок отправляет запросы загрузчику, проходя через промежуточное ПО загрузчика. После завершения загрузки страницы загрузчик генерирует ответ и отправляет его обратно движку, также через промежуточное ПО загрузчика. Движок передаёт ответ пауку для обработки, пропуская его через промежуточное ПО паука. Паук обрабатывает ответ и возвращает извлечённые элементы и новые запросы движку. Затем движок отправляет обработанные элементы в конвейеры элементов, а обработанные запросы — в планировщик, после чего процесс повторяется с шага получения следующего запроса, пока планировщик не вернёт пустой результат.

Центральным элементом любого проекта Scrapy является паук (Spider) — класс, который определяет, какие запросы отправлять и как разбирать их ответы. Пауки должны наследоваться от базового класса scrapy.Spider и определять уникальное имя, а также метод start(), который возвращает начальные запросы, и метод parse(), принимающий ответ и извлекающий из него данные. Внутри метода parse() для извлечения данных из HTML или XML используются селекторы Scrapy, предоставляющие мощный и удобный API для запросов к DOM с помощью CSS или XPath, извлечения атрибутов, установки значений по умолчанию и даже применения регулярных выражений в рамках одной цепочной конструкции.

Default structure of Scrapy projects

Before delving into the command-line tool and its sub-commands, let's first understand the directory structure of a Scrapy project.

Though it can be modified, all Scrapy projects have the same file structure by default, similar to this:

```
scrapy.cfg
myproject/
  __init__.py
  items.py
  middlewares.py
  pipelines.py
  settings.py
  spiders/
    __init__.py
    spider1.py
    spider2.py
    ...
```

The directory where the `scrapy.cfg` file resides is known as the **project root directory**. That file contains the name of the python module that defines the project settings. Here is an example:

```
[settings]
default = myproject.settings
```

Selectors

When you're scraping web pages, the most common task you need to perform is to extract data from the HTML source. There are several libraries available to achieve this, such as:

- BeautifulSoup** is a very popular web scraping library among Python programmers which constructs a Python object based on the structure of the HTML code and also deals with bad markup reasonably well, but it has one drawback: it's slow.
- lxml** is an XML parsing library (which also parses HTML) with a pythonic API based on **ElementTree**. (lxml is not part of the Python standard library.)

Scrapy comes with its own mechanism for extracting data. They're called selectors because they "select" certain parts of the HTML document specified either by **XPath** or **CSS** expressions.

XPath is a language for selecting nodes in XML documents, which can also be used with HTML. **CSS** is a language for applying styles to HTML documents. It defines selectors to associate those styles with specific HTML elements.

Spiders

Spiders are classes that define how a site, or a group of sites, is scraped: which requests to send, and how to parse their responses to extract data and to send additional requests.

A crawl goes as follows:

- Scrapy iterates the `start()` method of the spider to get the initial requests. By default, that method yields a **Request** object for each URL in `start_urls`, with `parse()` as **callback**.
- Scrapy downloads each request and calls its callback with the resulting **Response**.
- Callbacks parse the response, typically using **Selectors**, and return or yield **item objects** with the extracted data and **Request** objects to continue the crawl, which go back to step 2. See **Callback output**.
- Items go through **item pipelines**, and are usually stored through **Feed exports**.

Scrapy includes different spider classes for different purposes, described below.

Для структурирования извлекаемых данных в Scrapy используются элементы (Items) — простые объекты типа «ключ-значение», которые определяют поля, подлежащие сбору. Элементы позволяют задавать схему данных и обеспечивают их согласованную обработку на всех последующих этапах, включая конвейеры и экспорт. Хотя элементы можно создавать как обычные словари Python, Scrapy предоставляет класс Item, который поддерживает определение полей и помогает избежать опечаток. Для заполнения элементов данными из селекторов часто применяются загрузчики элементов (Item Loaders), которые позволяют описывать правила извлечения и обработки для каждого поля, делая код паука более декларативным и удобным для сопровождения.

Items

The main goal in scraping is to extract structured data from unstructured sources, typically, web pages. *Spiders* may return the extracted data as *items*, Python objects that define key-value pairs.

Scrapy supports [multiple types of items](#). When you create an item, you may use whichever type of item you want. When you write code that receives an item, your code should [work for any item type](#).

Item Types

Scrapy supports the following types of items, via the `itemadapter` library: [dictionaries](#), [Item objects](#), [dataclass objects](#), [attrs objects](#) and [Pydantic models](#).

Dictionaries

As an item type, `dict` is convenient and familiar.

Item objects

`Item` provides a `dict`-like API plus additional features that make it the most feature-complete item type: приоритета его выполнения.

Настройки (Settings) в Scrapy конфигурируют практически все аспекты работы фреймворка: какие промежуточные ПО и расширения включены и в каком порядке, как ведут себя конвейеры элементов, куда экспортируются данные и какие значения по умолчанию используются. Настройки обычно задаются на уровне проекта в файле `settings.py`, но каждый паук может переопределить их для себя через атрибут `custom_settings`, что позволяет, например, включить определённое промежуточное ПО только для тех пауков, которым оно действительно необходимо. Для управления проектом и его компонентами используется командная строка `scrapy`, предоставляющая набор команд как для работы внутри проекта, так и для глобальных операций. Глобальные команды, такие как `startproject`, `genspider`, `settings` и `shell`, позволяют создавать проект, генерировать шаблоны пауков и тестировать селекторы в интерактивной среде, а проектные команды, например `crawl`, `check`, `list` и `parse`, предназначены для запуска и отладки пауков внутри конкретного проекта. Одной из ключевых возможностей Scrapy является встроенная система экспорта данных (Feed Exports), позволяющая сохранять извлечённые элементы в файлы или хранилища без написания дополнительного кода. Scrapy поддерживает сериализацию в форматы JSON, CSV, XML и другие, а также различные бэкэнды хранения, включая локальные файлы, FTP, Amazon S3 и Google Cloud Storage. Настройка экспорта осуществляется через параметр `FEEDS` в файле настроек, где указывается URI назначения, формат сериализации и дополнительные параметры, такие как кодировка или перезапись существующих файлов. Это позволяет быстро получить результаты обхода в удобном формате, не прибегая к созданию конвейеров элементов.

Feed exports

One of the most frequently required features when implementing scrapers is being able to store the scraped data properly and, quite often, that means generating an "export file" with the scraped data (commonly called "export feed") to be consumed by other systems.

Scrapy provides this functionality out of the box with the Feed Exports, which allows you to generate feeds with the scraped items, using multiple serialization formats and storage backends.

This page provides detailed documentation for all feed export features. If you are looking for a step-by-step guide, check out [Zyte's export guides](#).

Serialization formats

For serializing the scraped data, the feed exports use the [Item exporters](#). These formats are supported out of the box:

- JSON
- JSON lines
- CSV
- XML

После того как паук извлёк элементы, они передаются в конвейеры элементов (Item Pipelines) — компоненты, которые обрабатывают элементы последовательно. Каждый компонент конвейера представляет собой Python-класс, реализующий простой метод, который получает элемент и может выполнить над ним такие операции, как очистка полей, проверка данных, удаление дубликатов или сохранение в базу данных. Конвейеры позволяют вынести постобработку данных за пределы паука, делая логику повторно используемой и легко настраиваемой через файл `settings.py` с указанием пути к классу и

Item Pipeline

After an item has been scraped by a spider, it is sent to the Item Pipeline which processes it through several components that are executed sequentially.

Each item pipeline component (sometimes referred as just "Item Pipeline") is a Python class that implements a simple method. They receive an item and perform an action over it, also deciding if the item should continue through the pipeline or be dropped and no longer processed.

Typical uses of item pipelines are:

- cleansing HTML data
- validating scraped data (checking that the items contain certain fields)
- checking for duplicates (and dropping them)
- storing the scraped item in a database

Writing your own item pipeline

Each item pipeline is a [component](#) that must implement the following method:

`process_item(self, item)`

This method is called for every item pipeline component.

`item` is an [item object](#), see [Supporting All Item Types](#).

`process_item()` must either return an [item object](#) or raise a `DropItem` exception.

Dropped items are no longer processed by further pipeline components.

Parameters: `item` ([item object](#)) – the scraped item

Scrapy также предоставляет богатую систему расширений (Extensions) и дополнений (Add-ons), которые позволяют подключать дополнительную функциональность, такую как автоматическое регулирование скорости запросов (AutoThrottle), управление файлами cookie, кэширование и многое другое. Архитектура фреймворка предусматривает гибкие механизмы для написания собственных расширений и промежуточных слоёв, что делает Scrapy не просто инструментом для сбора данных, а полноценной платформой для построения масштабируемых и поддерживаемых краулеров.

Extensions

Extensions are [components](#) that allow inserting your own custom functionality into Scrapy.

Unlike other components, extensions do not have a specific role in Scrapy. They are "wildcard" components that can be used for anything that does not fit the role of any other type of component, such as collecting stats or enforcing a memory limit; see [Scrapy's building blocks](#) for a rundown of other alternatives.

Loading and activating extensions

Extensions are loaded at startup by creating a single instance of the extension class per spider being run.

To enable an extension, add it to the `EXTENSIONS` setting. For example:

```
EXTENSIONS = {
    "scrapy.extensions.corestats.CoreStats": 500,
    "scrapy.extensions.telnet.TelnetConsole": 500,
}
```

`EXTENSIONS` is merged with `EXTENSIONS_BASE` (not meant to be overridden), and the priorities in the resulting value determine the *loading order*.

As extensions typically do not depend on each other, their loading order is irrelevant in most cases. This is why the `EXTENSIONS_BASE` setting defines all extensions with the same order (`0`). However, you may need to carefully use priorities if you add an extension that depends on other extensions being already loaded.